

Progettazione di Sistemi Digitali – AA 2025/26

Relazione di progetto – Radar a Ultrasuoni

Gabriele Campo, Alessio Fontana, e Matteo Martellini

(Data: 13 gennaio 2026)

Abstract. Nel presente lavoro è stata progettata e realizzata una piattaforma digitale basata su FPGA per l'implementazione di un radar ad ultrasuoni, interamente descritta in linguaggio Verilog HDL e validata su scheda Terasic DE10-Lite. Si descriverà l'architettura e si concluderà con i risultati di sintesi e di fitting, l'analisi delle risorse impiegate e la verifica temporale del progetto, con particolare attenzione all'individuazione del cammino critico.

Indice

1	Introduzione	1
2	Metodologia di Progetto	2
3	Risorse hardware utilizzate	2
3.1	Risorse on-board	2
3.2	Componenti esterni	3
4	Descrizione Architetture	3
4.1	Motore	3
4.2	Sensore	4
4.2.1	Echo	4
4.2.2	Enabler e SR	5
4.2.3	Filtro mediano	5
4.2.4	Display a 7-segmenti	5
4.3	VGA	6
4.3.1	Temporizzazione e scansione dello schermo	6
4.3.2	Gestione grafica e rappresentazione del radar	6
4.3.3	Divisore di frequenza e generazione del clock di abilitazione	7
4.3.4	Look-up table	8
4.3.5	Memoria ROM	8
5	Verifica funzionale e testbench	8
6	Sintesi, fitting e analisi temporale	9
6.1	Sintesi	9
6.2	Fitting	9
6.3	Analisi temporale	9
7	Analisi dei warning	10
8	Commenti e conclusioni generali sul progetto	10
	Riferimenti bibliografici	11

1. INTRODUZIONE

Lo scopo del progetto affrontato consiste nella realizzazione, mediante piattaforma **FPGA**, di un radar ad ultrasuoni, ovvero di una versione semplificata del noto sistema di rilevamento di distanze basato su onde radio. Il progetto è stato interamente sviluppato in linguaggio *HDL Verilog* utilizzando l'ide **Quartus Prime**^[1], prodotto da **Altera**, ed è stato infine validato su una FPGA Altera della serie MAX 10 presente sulla scheda di sviluppo Terasic DE10-Lite^{[4][5]}.

Pur nella sua semplicità, l'implementazione proposta vuole garantire un alto grado di affidabilità,

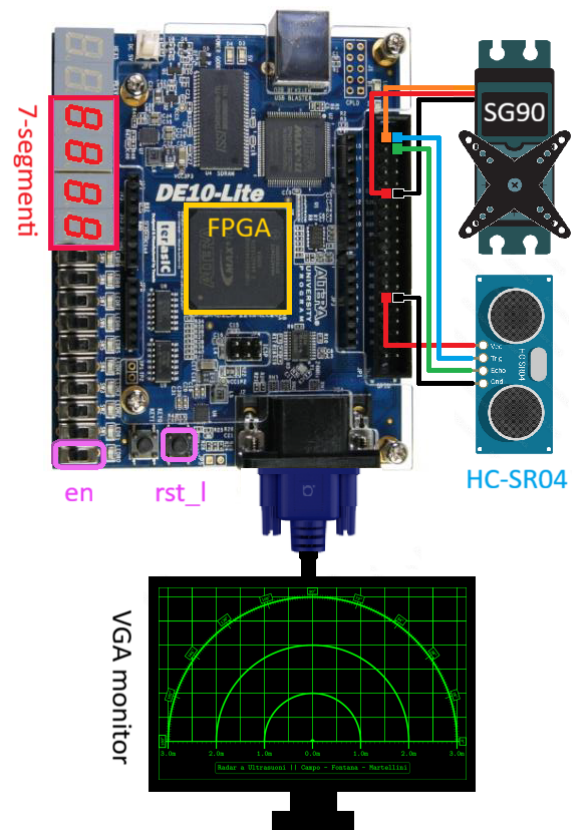


Figura 1 Schema riassuntivo del sistema e delle periferiche utilizzate. Connessioni con GPIO: **Vcc** (+3.3V/+5V), gnd, **pwm**, **Trig**, **Echo**

preservando interamente le funzionalità base di un comune radar. Per questo motivo, oltre ai blocchi di pilotaggio del servo motore, acquisizione e gestione dei dati provenienti dal sensore a ultrasuoni, il sistema integra un modulo per il filtraggio dei dati basato su filtro mediano. Ciò permette di migliorare la qualità delle informazioni riportate all'utente, attuando una reiezione degli *outlier* dovuti alla presenza di disturbi acustici.

Per una migliore esperienza di utilizzo, inoltre, sono state aggiunte un'unità per la generazione dei segnali di controllo di un monitor VGA, sul quale visualizzare graficamente gli oggetti rilevati, e un'unità di gestione del display a sette segmenti presente sulla scheda, utile alla rappresentazione numerica delle di-

stanze misurate. Uno schema riassuntivo del sistema complessivo può essere consultato in Fig.1, dove sono evidenziate le principali interconnessioni tra la piattaforma FPGA e i componenti hardware utilizzati per la realizzazione del progetto.

In questa relazione viene anzitutto illustrata l'architettura del sistema digitale progettato, costituita da diversi blocchi funzionali. A seguire sono riportate le descrizioni dettagliate di ciascun blocco e discusse le modalità di descrizione utilizzate, facendo sempre riferimento alle strategie di progetto adottate. Le varie fasi di sviluppo non sono state prive di criticità, affrontate mediante verifiche funzionali e testbench, che hanno garantito una piena comprensione dei limiti comportamentali del sistema ed il raggiungimento del risultato finale.

2. METODOLOGIA DI PROGETTO

Il sistema oggetto del presente lavoro è stato progettato secondo i criteri del **progetto digitale sincrono statico**. L'intera architettura è basata su un unico dominio di clock, comune a tutti i moduli che compongono il progetto, quello di sistema. Tale scelta consente di semplificare l'analisi temporale, ridurre le criticità legate alla sincronizzazione tra domini differenti e migliorare la robustezza complessiva del sistema.

Il segnale di clock viene distribuito uniformemente a tutti i blocchi logici, evitando deliberatamente l'uso di clock *derivati* o *divisi*. La variazione della frequenza di operazione dei diversi sottosistemi non viene ottenuta tramite la generazione di nuovi segnali di clock, bensì attraverso l'impiego di segnali di abilitazione. In questo modo, ogni modulo può operare a una frequenza effettiva inferiore rispetto a quella del clock principale, pur rimanendo sincronizzato allo stesso riferimento temporale *globale*. Questo approccio è preferibile in ambito FPGA, poiché evita problemi di skew, glitch e asincronia associati alla gestione di più clock.

All'interno dei moduli è stata mantenuta una chiara distinzione tra la fase operativa e la fase di mantenimento dello stato. Quando il segnale di abilitazione - locale - è attivo, il registro o la macchina a stati aggiorna il proprio contenuto in base alla logica combinatoria associata; in caso contrario, lo stato viene mantenuto invariato. Tale comportamento è riconducibile al modello di flip-flop di tipo D con ingresso di abilitazione - **E flip-flop** -, primitiva estremamente utile nell'ambito del progetto sincrono statico.

I segnali globali **enable** e **reset_1** - reset attivo basso - sono per loro natura esterni al sistema digitale, perciò potenzialmente asincroni rispetto al clock principale; infatti, provengono da periferiche controllate dall'utente umano, quali *slide switch* e *push button*. Poiché la propagazione diretta di segnali asincroni può introdurre fenomeni di metastabilità casuali e incontrollati in tutto il sistema, tali

segnali non sono impiegati direttamente nella logica sequenziale. Al contrario, essi sono sincronizzati in un unico punto del sistema, mediante un sincronizzatore ottimo, e poi propagati. Il sincronizzatore ottimo adottato, schematizzato in Fig. 2, è costituito da una catena di due flip-flop di tipo D connessi in cascata e pilotati dal clock a 50 MHz. Questa struttura riduce in modo significativo la probabilità che una transizione asincrona provochi metastabilità nei registri interni. In particolare, il primo flip-flop assorbe l'eventuale metastabilità, mentre il secondo fornisce un segnale decisamente più stabile e affidabile alla logica successiva. A valle di questi due sincronizzatori, i segnali **enable** e **reset_1** assumono il ruolo di segnali globali **sincroni**, condivisi perciò da tutti i moduli del sistema.

L'adozione di queste metodologie di progetto ha permesso di focalizzare l'attenzione sui requisiti di funzionalità del sistema, senza doversi preoccupare di criticità legate alla temporizzazione e alla sincronizzazione dei segnali.

3. RISORSE HARDWARE UTILIZZATE

Il progetto fa uso in maniera eterogenea di risorse hardware integrate sulla scheda di sviluppo e di componenti esterni interfacciati ad essa tramite i pin GPIO o il connettore VGA.

3.1. Risorse on-board

Tra le risorse on-board rientrano, escludendo la piattaforma FPGA:

- Display a sette segmenti: impiegato per la visualizzazione numerica (in millimetri) delle distanze misurate;
- Slide-switch: impiegato per il controllo del segnale di **enable**;
- Push-button: impiegato per la generazione del segnale di **reset₁**;
- DAC a 4-bit: impiegato per eseguire la conversione dei segnali digitali a 4-bit VGA_R, VGA_G e VGA_B necessaria al trasferimento via VGA.

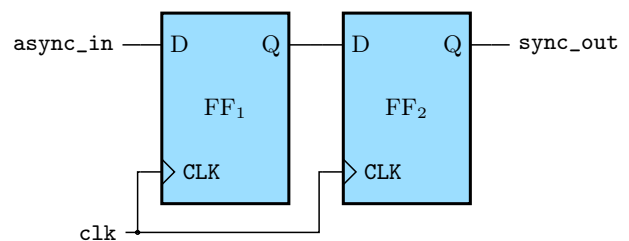


Figura 2 Sincronizzatore ottimo a due flip-flop D per la sincronizzazione di segnali esterni asincroni al dominio di clock del sistema.

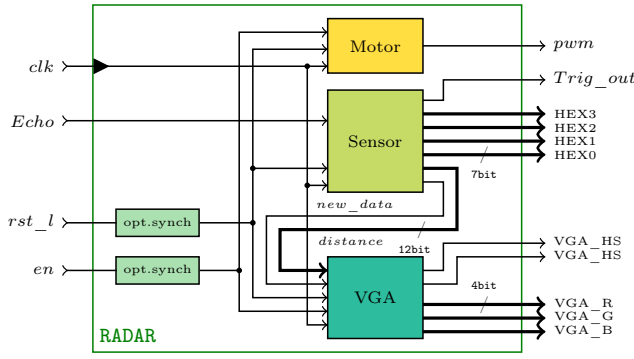


Figura 3 Diagramma a blocchi funzionali dell'architettura del sistema radar con porte di i/o.

3.2. Componenti esterni

Per quanto riguarda le periferiche esterne, sono stati utilizzati:

- Sensore a ultrasuoni HC-SR04^[3]: utilizzato per la misura della distanza, tramite tempo di volo, dell'oggetto rilevato;
- Servomotore SG90: impiegato per la rotazione del sensore e la scansione dell'area di interesse su un intervallo angolare compreso tra 0° e 180°;
- Monitor VGA: utilizzato per la visualizzazione grafica del radar, con una griglia di riferimento per la stima delle distanze e la rappresentazione dell'oggetto rilevato tramite un indicatore di colore rosso.

4. DESCRIZIONE ARCHITETTUALE

Il sistema digitale progettato può essere rappresentato nella sua interezza dal diagramma a blocchi riportato in Fig.3. Esso riceve in ingresso due segnali asincroni esterni provenienti da uno slide-switch e da un push-button, il segnale di echo generato dal sensore ad ultrasuoni e il segnale di clock. In uscita, invece, fornisce il segnale pwm di controllo posizionale del servo-motore, il segnale di trigger del sensore, i segnali di gestione del display a sette segmenti e i segnali utili al pilotaggio dell'interfaccia VGA.

Si possono distinguere tre blocchi funzionali principali, tralasciando la gestione dei segnali asincroni¹. Seguendo la strategia del partizionamento gerarchico, il sistema è stato progettato distinguendo descrizioni funzionali relative alle diverse aree operative.

localparam	Value	T[s]	Angolo
MIN_DUTY	25000	0.5 ms	-90°
MID_DUTY	75000	1.5 ms	0°
MAX_DUTY	125000	2.5 ms	+90°
DELTA	400	8 μs	0.72°

Tabella I Parametri per il controllo del *duty-cycle* e la conseguente gestione del servomotore.

4.1. Motore

Innanzitutto vengono descritte le caratteristiche del motore. Esso presenta tre fili di connessione - 5V, gnd e pwm - ed è in grado ruotare di $\pm 90^\circ$ rispetto ad una posizione di riferimento. L'angolo di destinazione è controllato dall'utente mediante la durata dell'impulso attivo alto a 50 Hz che il motore riceve sull'ingresso pwm. Vi è una relazione lineare tra la durata dell'impulso e la posizione del motore.

Per riprodurre il movimento di un classico radar², abbiamo dovuto generare un segnale *pwm* a 50 Hz con larghezza dipendente dal tempo. È stato utilizzato un contatore a 20-bit, il cui valore è incrementato ad ogni ciclo di clock a 50 MHz e confrontato con il valore del segnale di medesima profondità *duty_cycle*. Quest'ultimo segnale è generato all'interno del modulo *dutycycle*, mentre il confronto avviene nel secondo e ultimo modulo dell'unità, *pwm*. In base all'esito del confronto, il segnale di uscita *pwm* viene alzato, abbassato o mantenuto. Quando il contatore giunge al suo valore limite³ viene azzerato e in concomitanza si genera un impulso di durata $1 T_{clk}$, denominato *tick_50hz*. Questo segnale viene portato nel modulo *dutycycle*, dove è utilizzato come abilitazione per l'aggiornamento del valore di *duty_cycle*, che in questo modo non varia in maniera "continua", ma solo ogni 20 ms, ovvero solo quando necessario.

Nel modulo di generazione del segnale *duty_cycle* sono definiti alcuni parametri locali, utili all'assegnazione del valore al segnale. In particolare sono definiti il valore minimo, il valore centrale, il valore massimo del segnale, il valore dell'incremento ed il numero di incrementi necessario a coprire completamente il settore angolare disponibile. I primi tre valori sono calcolati conoscendo la relazione tra larghezza dell'impulso e angolo, e ricordando che lavoriamo con un clock a 50 MHz, mentre *DELTA* e *STEPS* dipendono da una scelta progettuale: vogliamo che il radar scandisca 180° in 5 s. Siccome il segnale di controllo *pwm* ha una frequenza di 50 Hz sono necessari $5 s \cdot 50 Hz = 250$ passi. Questi parametri, insieme ai corrispondenti valori di durata dell'impulso e angolo, sono riportati in Tab. I. Si ricorda che:

¹ Come detto, essi vengono resi sincroni da sincronizzatori ottimi prima di essere distribuiti a tutto l'apparato.

² Sweep continuo da 0° a 180° e ritorno, senza arresti e ripetuto ciclicamente

³ Valore pari a 1000000, scelto perché corrispondente a 20 ms, periodo del segnale di uscita *pwm*

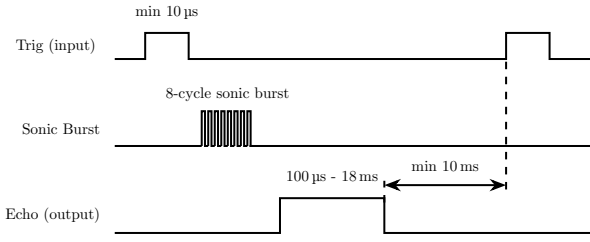


Figura 4 Rappresentazione dei tempi caratteristici di funzionamento del sensore HC-SR04

$$\text{DELTA} \cdot \text{STEPS} = \text{MAX_DUTY} - \text{MIN_DUTY}$$

L'assegnazione ad ogni tick a 50 Hz avviene a valle di un sommatore che incrementa ad ogni step il valore in uscita. Una volta eseguiti 250 passi il contatore di appoggio viene azzerato e il segnale a 1-bit relativo al verso di rotazione viene invertito. Si passa ad un altro stato e per i 250 passi successivi, dunque, il valore di `duty_cycle` verrà decrementato fino a tornare al minimo.

È importante rimarcare che, come illustrato nella Sez.2, quando l'enable o `tick_50hz` sono bassi si entra in fase di *mantenimento*: tutte le uscite e i segnali interni vengono conservati.

Entrambi i moduli di cui è formato questo blocco sono dotati di reset - sincrono attivo basso - ed enable. Quando l'enable è basso, il motore si ferma; quando invece è il reset ad essere basso, il motore torna alla posizione di partenza, ovvero quella relativa a -90° .

4.2. Sensore

Si prosegue con l'analisi del blocco che permette la rilevazione della distanza: il sensore HC-SR04. Quest'ultimo presenta 4 pin - Vcc, gnd, trigger ed echo - e proprio attraverso il segnale inviato da trigger e quello ricevuto da echo è possibile ricavare la distanza dell'oggetto, a partire dalla velocità del suono e dal tempo di volo.

L'impulso di trigger ha una durata di 10 µs e il tempo che intercorre tra un impulso e l'altro è di 100 ms⁴. Per generare l'impulso è stato utilizzato un contatore a 23-bit, il cui valore è incrementato ad ogni ciclo di clock a 50 MHz: da 0 a 500 - corrispondenti a 10 µs - `trig` viene mantenuto alto per poi esser tenuto basso fino al valore limite⁵. Ricepito l'impulso di trigger, il sensore risponde emettendo un treno di 8 impulsi acustici a 40 kHz dall'emettitore; questo ri-

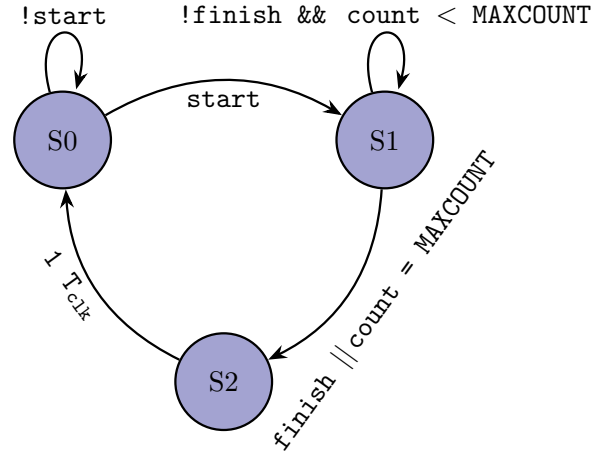


Figura 5 Diagramma a palle della FSM utilizzata nel blocco echo

mane poi in ascolto, attendendo il ritorno, o meno⁶, di questi impulsi a ultrasuoni. La distanza è allora proporzionale alla durata del segnale di `echo`, ovvero all'intervallo temporale intercorso tra l'emissione e la ricezione degli ultrasuoni. Una rappresentazione temporale dei tre segnali è riportata in Fig. 4

4.2.1. Echo

Il segnale di `echo` è gestito attraverso una macchina a stati, il cui diagramma a palle è riportato in Fig. 5. Nello stato `S0`, il sensore rimane in attesa che venga rilevata la presenza del segnale `start`, definito come il rilevatore del fronte in salita del segnale `echo`. Rilevato il segnale di inizio la macchina passa allo stato `S1`, nel quale rimane fino all'arrivo del segnale `finish` o il raggiungimento del valore massimo `MAXCOUNT`⁷ da parte del contatore. Infine, nello stato `S2` - nel quale la macchina entra raggiunto `MAXCOUNT` o una volta arrivato il segnale `finish` - viene memorizzato il conteggio e attivato un segnale di enable, usato poi dagli stadi successivi `enabler` ed `SR`.

Attraverso il numero memorizzato è facile ricavare la distanza, in millimetri, dell'oggetto rilevato, sapendo che il conteggio è stato effettuato a 50 MHz, quindi secondo la formula:

```
1 distance <= (dis_reg * 226) >> 16
```

Questa operazione implementa la classica formula per il calcolo della distanza⁸:

$$\text{distanza} = \frac{\text{velocità del suono} \times t}{2}$$

⁶ Dopo una durata di 36 ms viene interrotta la ricezione, indicando che non sono stati rilevati oggetti nel range di corretto utilizzo del sensore.

⁷ Il valore `MAXCOUNT` è pari a 441428, corrispondente a circa 3 m, considerando la velocità del suono di 340 ms^{-1} e frequenza di clock di 50 MHz.

⁸ Il fattore $1/2$ serve per considerare che il tempo di volo comprende andata e ritorno del suono.

⁴ Il gap di 100 ms è necessario per garantire la corretta separazione tra successivi segnali di echo. Si tiene conto del fatto che per una misura **non** valida il sensore risponde con un impulso di 36 ms

⁵ Valore pari a 5000000, numero corrispondente a 100 ms, con il clock a 50 MHz.

Possiamo notare come nella versione implementata in Verilog non sia stata effettuata una moltiplicazione per la costante 0.00345 - o una divisione per 290 - ma una moltiplicazione per un intero positivo seguita da uno shift logico di 16-bit, corrispondente ad una divisione per 2^{16} . In questo modo vengono utilizzate meno risorse, riducendo anche l'impatto sul percorso critico.

4.2.2. Enabler e SR

Il segnale **en_sr**, in uscita dal blocco *echo*, pilota i blocchi *enabler* ed *SR*. Il blocco *enabler* utilizza un contatore per generare un segnale di abilitazione verso il filtro mediano: l'uscita **en** si alza per un ciclo di clock ogni qualvolta sono stati conteggiati 5 impulsi validi di **en_sr**, indicando che sono stati ricevuti 5 nuovi campioni di distanza misurata. Contestualmente, il blocco *SR* (Shift Register) sfrutta **en_sr** come abilitazione per lo scorrimento dei dati: ad ogni attivazione, il nuovo dato in ingresso viene memorizzato e quelli precedenti scalano di una posizione. Così facendo, si trovano in uscita gli ultimi 5 campioni acquisiti, pronti per essere elaborati dal filtro.

4.2.3. Filtro mediano

Il filtro mediano è adibito alla reiezione dei valori anomali - outliers⁹ - che si presentano durante la rilevazione della distanza, fornendo così in uscita un valore stabile e significativo. È stato implementato attraverso una macchina a stati - a 7 stati (S0...S6) - che permette di applicare un algoritmo di sorting, attraverso confronti e scambi, sui dati in ingresso. Nello stato S0 vengono caricati i dati, ovvero le 5 distanze misurate, che verranno successivamente ordinate negli stati S1, S2, S3, S4 ed S5; l'ordinamento attuato fa sì che in terza posizione si trovi la mediana delle misure. Infine, nello stato S6 viene assegnata all'uscita del filtro il valore del terzo registro di appoggio, contenente la mediana. L'elaborazione richiede 7 cicli di clock, per questo motivo è stato utilizzato un *ritardatore* per trasformare **en_mf** in un segnale **new_data** sincronizzato con il segnale **distance** in uscita dal blocco funzionale corrente. Questo concetto è particolarmente importante ai fini della rappresentazione su monitor VGA della distanza misurata, e verrà compreso appieno nel Par. 4.4.3.4.3.2.

⁹ Gli outliers sono causati da vari fattori, come l'assorbimento del segnale ad ultrasuoni, che pertanto non verrà correttamente rilevato, o la presenza di interferenze o riflessioni.

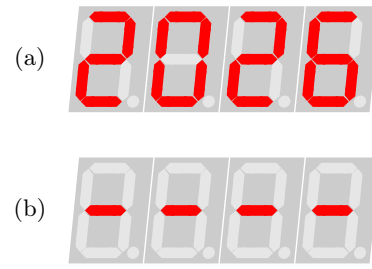


Figura 6 Rappresentazione del funzionamento dei display a 7-segmenti: (a) Misura valida, (b) Misura fuori range

4.2.4. Display a 7-segmenti

Per avere una lettura immediata della distanza sono stati utilizzati i display a 7-segmenti presenti sulla scheda. In particolare ne sono stati utilizzati 4, con la finalità di rappresentare, in base decimale, la distanza rilevata in millimetri e, nei casi di misura fuori range, accendere solo i segmenti orizzontali, come mostrato in Fig. 6.

La conversione dei numeri da binario - a 12 bit - a notazione BCD - a 16 bit - è stata fatta sfruttando l'algoritmo Double Dabble. L'algoritmo riceve un dato binario a 12 bit e lo converte in formato BCD a 16 bit, così da rappresentare separatamente migliaia, centinaia, decine e unità. La conversione avviene esaminando il registro suddiviso in gruppi da 4 bit: quando il valore di un gruppo supera il valore 4, in decimale, viene applicata una correzione sommando 3. Questa operazione, è seguita da uno scorrimento a sinistra, in modo da assicurare la corretta propagazione dei riporti tra le cifre decimali.

La verifica della bontà della distanza misurata è affidata al blocco *validator*. Quest'ultimo riceve in ingresso la misura - in binario - e la confronta con i parametri MAXDIST e MINDIST¹⁰, se questa risulta appartenente all'intervallo di validità **valid** viene messo a 1 altrimenti viene abbassato a 0.

Il segnale **valid** pilota il multiplexer del blocco *mux_valid*, il quale porta in uscita il valore della distanza misurata - espresso in standard BCD - se **valid** è alto.

Il blocco *mux_valid* pilota i quattro decoder relativi ai display a 7-segmenti adibiti alla visualizzazione della misura effettuata e filtrata. Il pilotaggio avviene attraverso un vettore a 7-bit che indica quali LED del display accendere; in particolare il codice 7'b0111111, corrispondente al solo trattino orizzontale centrale, è usato per indicare che la misura è fuori range, come mostrato in Fig. 6. Il pilotaggio del singolo display è illustrato sul manuale della scheda^[5].

¹⁰ MAXDIST e MINDIST sono qui espressi in decimale e valgono rispettivamente 3 m e 3 cm, rispettando quindi il range di corretto funzionamento del sensore.

Il segnale **valid** è usato anche per pilotare il blocco *mux_valid_12bit*, che permette l'uscita - verso il modulo **VGA** - del segnale a 12-bit relativo alla distanza misurata, rappresentata in millimetri. Se alto, questo segnale permette l'uscita del dato; se basso, esso vincola l'uscita a zero¹¹.

4.3. VGA

Il sistema **VGA** realizzato è strutturato attorno a due moduli principali, affiancati da alcuni blocchi ausiliari necessari al corretto funzionamento dell'intero progetto.

Il modulo *vga_controller* si occupa della temporizzazione del segnale **VGA**, gestendo la scansione dei pixel dello schermo e la generazione dei segnali di sincronismo orizzontale e verticale - **hsync** e **vsync**. In altre parole, questo modulo definisce quando ciascun pixel deve essere disegnato. Il modulo *vga_graphics* si occupa della gestione dei segnali RGB, decidendo cosa visualizzare per ogni pixel visibile: esso stabilisce se mostrare lo sfondo oppure rappresentare il bersaglio individuato dal sistema radar, calcolandone la posizione a partire dai dati provenienti dai moduli esterni.

Accanto a questi, il sistema include un divisore di frequenza, due look-up table, contenenti i valori di seno e coseno relativi a un insieme di angoli prestabiliti, e una memoria ROM, nella quale viene caricato un file in formato .mif rappresentante lo sfondo.

4.3.1. Temporizzazione e scansione dello schermo

Il modulo *vga_controller* è responsabile della gestione della temporizzazione del protocollo **VGA**. È stata adottata la modalità **VGA** 640×480 a 60Hz, nella quale ogni frame è composto da 525 righe e ciascuna riga richiede 800 cicli di clock per essere disegnata. Di queste, solo 480 righe e 640 cicli corrispondono all'area effettivamente visibile sullo schermo¹², mentre le restanti porzioni sono dedicate ai segnali di sincronismo e alle pause di *front porch* e *back porch*, necessarie per il corretto funzionamento del monitor. Una schematizzazione del suddetto partizionamento è visibile in Fig. 7.

La scansione dello schermo è realizzata tramite due contatori sincroni: un contatore orizzontale **h_count**, che avanza a ogni ciclo di clock e scorre i pixel all'interno della riga, e un contatore verticale **v_count**, che incrementa al termine di ciascuna riga,

consentendo di passare alla riga successiva. Quando il contatore verticale raggiunge il valore massimo, entrambi i contatori vengono azzerati e inizia la scansione di un nuovo frame. A partire dai valori di questi contatori vengono generati i segnali di sincronismo **hsync** e **vsync**, che vengono portati a livello basso esclusivamente durante gli intervalli di sync pulse orizzontale e verticale, in conformità con lo standard **VGA**, che prevede segnali attivi bassi. L'attività di questi segnali è riportata in Fig. 8.

Il modulo produce inoltre il segnale **video_on**, che indica quando il pixel corrente si trova all'interno dell'area visibile dello schermo. Questo segnale è fondamentale poiché i segnali RGB devono essere considerati validi solo durante tale intervallo; quando **video_on** è basso, il segnale video viene disattivato, realizzando la fase di blanking. Quando **video_on** è alto, i valori di **h_count** e **v_count** rappresentano direttamente le coordinate X e Y del pixel corrente.

4.3.2. Gestione grafica e rappresentazione del radar

Il modulo *vga_graphics* utilizza le informazioni fornite dal *vga_controller* per generare i segnali RGB da inviare al monitor. Il suo compito principale è quello di disegnare lo sfondo e rappresentare graficamente il bersaglio rilevato dal radar, simulando al contempo la scansione angolare del sensore. Per quanto riguarda lo sfondo, il modulo legge una memoria ROM¹³ nella quale è memorizzata un'immagine monocromatica. L'accesso alla memoria avviene generando l'indirizzo corrispondente al pixel corrente mediante un'operazione di linearizzazione da due dimensioni a una dimensione. L'immagine è infatti memorizzata riga per riga: a ogni riga dell'immagine corrisponde una sequenza di celle di memoria pari alla larghezza dell'immagine. Il pixel in posizione (x, y) viene quindi individuato sommando al nume-

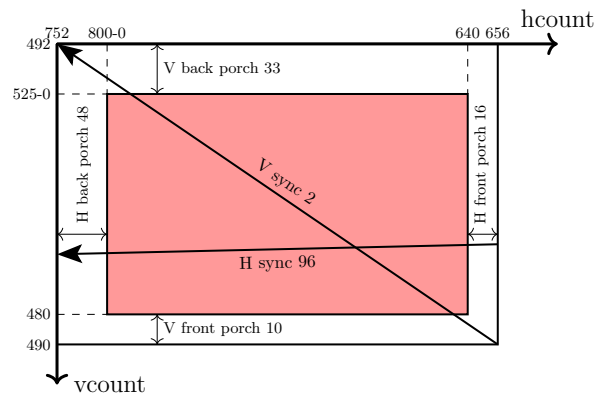


Figura 7 Schema temporale del segnale **VGA** su display 640×480

¹¹ Interpretato poi dalla logica del blocco **VGA** come una misura non valida, perché fuori range.

¹² Il periodo del segnale **VGA** ha origine nell'angolo in alto a sinistra dell'area video; di conseguenza, i primi 640 degli 800 cicli orizzontali e le prime 480 delle 525 righe verticali sono dedicati all'area visibile.

¹³ Lo sfondo viene caricato dalla ROM per ridurre il carico computazionale, evitando il ricalcolo tramite equazioni.

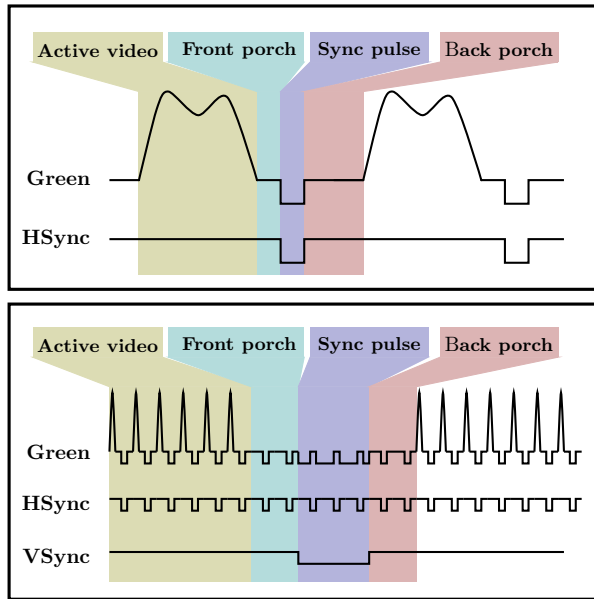


Figura 8 TempORIZZAZIONE del protocollo VGA

ro di pixel delle righe precedenti l'indice del pixel all'interno della riga corrente.

```
1 rom_addr <= (v_count * IMG_W) + h_count
```

Poiché la dimensione dell'immagine di sfondo non coincide perfettamente con l'area visibile dello schermo, viene verificato che il pixel corrente rientri nell'intervallo valido; in caso contrario, viene utilizzato un indirizzo di default.

La rappresentazione del bersaglio avviene a partire dai dati forniti dal modulo del sensore. Il segnale **new_data** indica la disponibilità di una nuova misura, prodotta dal filtro mediano con una frequenza di 2 Hz - ogni 0.5 secondi. Poiché questo segnale può rimanere alto per più cicli di clock, viene utilizzata una logica di rilevamento del fronte di salita, implementata tramite i registri **nd_reg1** e **nd_reg2**, per trasformarlo in un evento valido per un solo ciclo, evitando di avanzare di più di un settore per misura. La distanza del bersaglio è fornita in millimetri dal segnale **distance**. Per semplicità, è stata adottata la relazione 1 pixel = 1 centimetro, introducendo un segnale di appoggio **distx** per effettuare la conversione¹⁴. La divisione per 10 è stata sostituita da un'operazione equivalente ma meno onerosa in termini di risorse impiegate e ritardi generati. Successivamente, le coordinate polari del bersaglio vengono convertite in coordinate cartesiane utilizzando i valori di seno e coseno letti dalle LUT. I risultati vengono sommati alla posizione del centro delle circonferenze tracciate; per la coordinata verticale è necessario introdurre un segno negativo, poiché la coordinata y

è incrementata dall'alto verso il basso. Lo shift aritmetico è utilizzato per compensare il fattore di scala introdotto nella rappresentazione intera dei valori trigonometrici.

La scansione angolare del radar è simulata gestendo opportunamente l'indice delle LUT di seno e coseno. A tale scopo vengono utilizzati due segnali: **add_lut**, che rappresenta il settore angolare corrente e coincide con l'indice delle LUT, e **lut_ref**, che funge da riferimento per determinare il verso della scansione. L'indice avanza progressivamente da 0 a 9 e successivamente decresce da 9 a 0, simulando la rotazione del sensore da 0° a 180° e viceversa. In caso di reset, la posizione del bersaglio viene forzata al centro del sistema di riferimento polare e l'indice delle LUT viene riportato al valore iniziale.

La parte finale del modulo *vga_graphics* stabilisce il colore da inviare al monitor per ciascun pixel. Quando **video_on** è attivo, il modulo confronta la posizione del pixel corrente con la posizione del bersaglio calcolata in precedenza. Se il pixel cade all'interno di una finestra quadrata 8×8 centrata sul bersaglio, esso viene colorato per rendere il bersaglio facilmente visibile: il colore è rosso nel caso di bersaglio valido, mentre diventa blu quando il bersaglio è fuori dal range operativo del radar, situazione nella quale il quadrato viene visualizzato al centro delle circonferenze. Se il pixel corrente non appartiene all'area del bersaglio, viene visualizzata l'immagine di sfondo. In questo caso i canali rosso e blu sono sempre azzerati, mentre il canale verde dipende dal valore **green_pix** letto dalla ROM; trattandosi di un singolo bit, esso determina se il pixel di sfondo debba essere visualizzato in verde o in nero, consentendo la rappresentazione di un'immagine monocromatica verde su sfondo nero. Quando **video_on** è disattivo il pixel si trova in una zona di blanking, tutti i canali RGB vengono forzati a zero e la ROM viene letta presso un indirizzo di sicurezza, corrispondente a un pixel nero.

4.3.3. Divisore di frequenza e generazione del clock di abilitazione

Il divisore di frequenza è stato introdotto per soddisfare il requisito di clock del modulo *vga_controller*. La scheda fornisce un clock principale a 50MHz, mentre lo standard VGA 640×480 a 60Hz richiede un clock di 25MHz per la corretta scansione dei pixel. Anziché generare un nuovo dominio di clock, è stata adottata una soluzione sincrona basata su un contatore che produce un segnale di abilitazione periodico. Il parametro F del modulo definisce ogni quanti cicli di clock deve comparire l'impulso di uscita; nel caso specifico, ponendo $F = 2$, il segnale di abilitazione **enclk_pix** viene attivato una volta ogni due cicli del clock a 50 MHz, producendo un segnale a 25MHz.

¹⁴ Il segnale **distx** è stato definito signed per compatibilità con altri segnali; la larghezza effettiva scelta è di 12 bit, anche se ne sarebbero bastati 9.

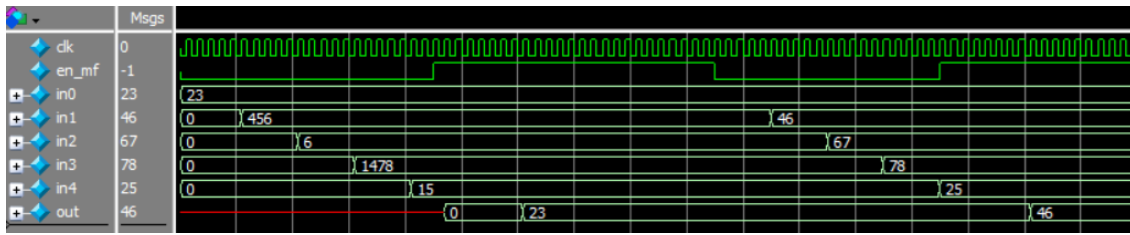


Figura 9 Forme d'onda e valori dei diversi segnali di input e output del modulo *median_filter*, simulati con la scrittura di un testbench. Si noti la discrepanza temporale tra il fronte dell'abilitazione *en_mf* e l'aggiornamento dell'uscita.

4.3.4. Look-up table

Le look-up table sono utilizzate per memorizzare i valori di seno e coseno relativi a un insieme discreto di angoli, necessari per la conversione delle coordinate polari del bersaglio in coordinate cartesiane. Il servo motore è stato pilotato in modo tale da compiere una rotazione di 180° in 5 secondi; poiché il filtro mediano produce un'uscita ogni 0,5 secondi, durante una rotazione completa si ottengono 10 campioni. Per questo motivo, l'intervallo angolare di 180° è stato suddiviso in 10 settori, e per ciascun settore sono stati pre-calcolati i corrispondenti valori di seno e coseno. Il radar non individua esattamente la posizione del bersaglio ma riconosce in quale settore angolare si trova e traccia la distanza per degli angoli predefiniti (9° , 27° , 45° ...) commettendo un errore angolare massimo di 9° . Dal momento che Verilog non supporta la rappresentazione in virgola mobile, tali valori sono stati codificati in formato fixed-point signed su 8 bit complessivi, di cui uno dedicato al segno. I valori reali sono stati moltiplicati per una potenza di due, arrotondati all'intero più vicino e successivamente convertiti in binario. Questa scelta introduce un errore massimo limitato, pari a circa lo 0,54%¹⁵ rispetto al valore iniziale. Le LUT sono state implementate come registri e non tramite le memorie dedicate della scheda per semplicità, in quanto lo spazio richiesto è estremamente ridotto.

4.3.5. Memoria ROM

La rappresentazione dello sfondo è affidata a una memoria ROM, implementata sfruttando i blocchi di memoria M9K interni alla FPGA MAX 10. L'uso di una ROM consente di evitare il calcolo dinamico dello sfondo tramite equazioni, riducendo in modo significativo la complessità computazionale del modulo grafico. La memoria è stata istanziata tramite l'IP Catalog di Quartus, selezionando il componente "ROM: 1-PORT" e configurandolo attraverso il wizard. Il contenuto della ROM è inizializzato mediante un file .mif, che contiene la codifica binaria

dell'immagine di sfondo. L'immagine originale è stata convertita in una sequenza di zeri e uni in un documento testuale, dove ciascun bit indica se il corrispondente pixel deve essere visualizzato oppure no, con un semplice codice in *python*. La conversione al formato .mif è stata eseguita usando un secondo programma. Per memorizzare l'intera immagine sono stati utilizzati 32 blocchi M9K¹⁶, affiancati da un numero limitato di LUT e registri per la gestione della logica di indirizzamento e dei dati. Durante il funzionamento, quando il segnale *video_on* è attivo, il pixel corrente viene sempre considerato valido; se i segnali RGB sono tutti a zero, il colore visualizzato è nero, il che può dare l'impressione di un pixel spento, pur trattandosi in realtà di un pixel correttamente visualizzato.

5. VERIFICA FUNZIONALE E TESTBENCH

La presenza di componenti esterni come il sensore ad ultrasuoni e il servomotore, che operano con input a bassa frequenza - 50 Hz - e in domini temporali non direttamente simulabili, ha reso irrilevante la realizzazione di testbench completi per l'intero sistema, la cui funzionalità è stata verificata direttamente sul prototipo. Di conseguenza, non sono stati eseguiti testbench significativi per l'architettura complessiva.

L'unico testbench rilevante riportato è quello che verifica il comportamento del filtro mediano. Questo ha permesso di verificare il corretto funzionamento della macchina a stati implementata per l'ordinamento dei campioni e di quantificare il numero di cicli di clock necessari all'elaborazione della mediana. Questo approccio ha fornito informazioni utili alla sincronizzazione del segnale *new_data* rispetto al blocco di acquisizione dei dati. La verifica è stata eseguita con il software Modelsim-Altera, ed il risultato è riportato in Fig. 9.

¹⁵ L'errore è riferito al valore trigonometrico e, attraverso le operazioni successive, può propagarsi e aumentare; tuttavia, l'unico scopo di queste operazioni è una visualizzazione grafica dal carattere assolutamente qualitativo.

¹⁶ Le word da memorizzare sono 254.562 (pari a 638×399); pertanto, è stata scelta una profondità di 262.144, corrispondente alla potenza di due più vicina.

6. SINTESI, FITTING E ANALISI TEMPORALE

Conclusa la fase di descrizione e progettazione dell'architettura del sistema, il progetto è stato sottoposto alle fasi di analisi, sintesi e fitting, finalizzate alla traduzione della descrizione HDL in una rete di porte logiche e registri e all'implementazione sulla piattaforma FPGA. Infine, è stata condotta un'accurata analisi temporale, volta a verificare il rispetto dei vincoli di timing e a individuare l'eventuale presenza di cammini critici, al fine di garantire il corretto funzionamento del sistema alla frequenza di clock prevista.

6.1. Sintesi

Durante questa fase l'architettura viene tradotta in una rappresentazione a livello **RTL**¹⁷ dal sintetizzatore. Il processo di sintesi produce una serie articolata di report informativi, che descrivono nel dettaglio la struttura logica ottenuta e le risorse hardware impiegate. Di questi, per brevità, riportiamo esclusivamente il **report summary** di sintesi, che fornisce una visione d'insieme delle risorse utilizzate durante l'elaborazione.

Total logic elements	1102
Total registers	381
Total memory bits	262144
Embedded multiplier 9-bit elements	7
I/O pins	48
Maximum fan-out node	MAX10_CLK1_50
Maximum fan-out	413
Average fan-out	3.29

Tabella II Analysis & Synthesis Summary

6.2. Fitting

Successivamente è stata eseguita la fase di fitting, comprendente le operazioni di *placement* e *routing*, necessarie all'allocazione delle risorse logiche e delle interconnessioni fisiche sul dispositivo target. Analogamente a quanto fatto per la sintesi, per motivi di brevità viene riportato esclusivamente il **report summary** del fitting, che consente di valutare in modo sintetico l'esito del processo di implementazione.

Total logic elements	1072 (2%)
Total registers	338
Total memory bits	262144 (16%)
Embedded multiplier 9-bit elements	7 (2%)
I/O pins	48 (13%)

Tabella III Fitter Summary

6.3. Analisi temporale

L'analisi temporale è stata effettuata per verificare che il progetto soddisfi i vincoli di timing imposti dal segnale di clock e che tutti i percorsi sincroni rispettino i requisiti di setup e hold, garantendo il corretto funzionamento del sistema.

È stato utilizzato un unico dominio di clock per tutto il sistema che è stato definito nel file .sdc implementando le seguenti constraints

```
1 create_clock -name MAX10_CLK1_50 -period
   20.000 -waveform {10 20} [get_ports {
   MAX10_CLK1_50}]
3 derive_clock_uncertainty
```

Listing 1 File .sdc relativo ai constraints di clock

Il clock imposto - denominato MAX10_CLK1_50 - ha allora un periodo di 20 ns.

A partire da questo clock, il tool di sintesi e timing analysis ha valutato i percorsi registro-logica-registro, individuando i peggiori ritardi associati alla logica combinatoria e alle interconnessioni interne al dispositivo, riportati in Tab. IV.

Worst-case Slack (setup)	8.306 ns
Worst-case Slack (hold)	0.148 ns
Worst-case F_{max}	85.51 MHz

Tabella IV Timing Analyzer Summary

Per ciascun percorso analizzato, il Launch Clock rappresenta il fronte di clock che raggiunge il registro di partenza, mentre il Latch Clock identifica il fronte di clock applicato al registro di arrivo. Inoltre, il Data Arrival corrisponde al tempo complessivo necessario affinché il dato, una volta lanciato dal registro sorgente, attraversi la logica e raggiunga il registro di destinazione. Il Data Required indica invece l'istante entro il quale il dato deve essere stabile per consentire un campionamento corretto, tenendo conto dei vincoli di setup del registro di arrivo.

La differenza tra il tempo richiesto e il tempo di arrivo del dato definisce lo Slack, parametro necessario per valutare il rispetto dei vincoli di timing.

In Fig. 10 è riportato il risultato dell'analisi effettuata; si osserva come il percorso critico presenti uno slack positivo pari a 8.306 ns. Questo indica che il dato arriva al registro di destinazione con un margine di sicurezza significativo rispetto all'istante in cui è richiesto.

Il timing analyzer fornisce anche la frequenza massima di clock, la quale risulta essere 85.51 MHz, oltre al percorso critico, a cui è associato lo slack minimo. Questo risulta partire dalla porta del filtro mediano `sensor:inst6|median_filter:inst7|out[5]` e arrivare alla porta del blocco di gestione grafica della VGA `vga:inst7|vga_graphics:inst10|distx[8]`, ed è il cammino che limita superiormente la frequenza massima di clock.

¹⁷ Register Transfer Level.

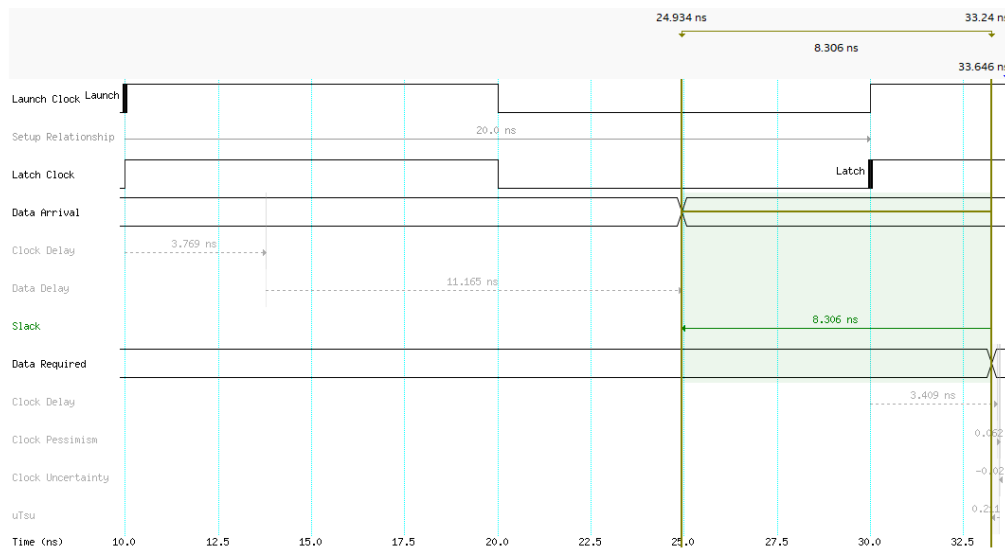


Figura 10 Diagramma di timing del percorso critico: rappresentazione dei segnali di Launch Clock e Latch Clock, del tempo di arrivo del dato (Data Arrival) e del tempo durante il quale i dati devono risultare stabili (Data Required). Il valore di slack positivo pari a 8.306 ns evidenzia il rispetto dei vincoli di timing e la presenza di un adeguato margine di sicurezza per il corretto campionamento dei dati.

7. ANALISI DEI WARNING

- *Warning del codice bcd2bin:* Risolto e commentato nel rispettivo modulo¹⁸.
- *Warning (10030): Net "lut.waddr_a" at cos_lut.v(7) has no driver or initial value, using a default initial value '0':* Compaiono tre di questi warning per la LUT contenute i valore del coseno (ed altri tre in maniera analoga per la LUT del seno), derivano dal fatto che Quartus non riconosce il template e mappa i valori sui singoli registri. Il problema può essere eliminato istanziando delle ROM, nello stesso modo in cui è stato fatto per il modulo rom.
- *Warning (10230): Verilog HDL assignment warning at echo.v(50): truncated value with size 32 to match size of target (20):* Numerosi warning di questo tipo sono legati al fatto che i numeri sono stati scritti senza specificare esplicitamente la dimensione e il formato (ad esempio, 1 invece di 1'b1), determinando troncamenti automatici per adattarsi alla larghezza del segnale di destinazione.
- *Warning (10905): Generated the EDA functional simulation netlist because it is the only supported netlist type for this device:* Questo warning è puramente informativo. Quartus genera una "netlist funzionale" per consentire la simulazione con strumenti EDA come ModelSim.

- *Warning (15714): Some pins have incomplete I/O assignments. Refer to the I/O Assignment Warnings report for details:* Questo warning è informativo e deriva dal fatto che non sono state definite constraint complete per le porte di I/O (direzione, pilotaggio e posizione fisica sulla scheda). Di conseguenza, nella finestra "Compilation Report", la sezione "Timing Analyzer – Unconstrained Paths – Summary" risulta evidenziata in rosso.
- *Warning (18236): Number of processors has not been specified which may cause overloading on shared machines:* Questo warning riguarda esclusivamente le prestazioni di compilazione e non il progetto hardware. Non essendo stato specificato il numero di core della CPU da utilizzare, Quartus sceglie automaticamente le risorse disponibili.
- *Warning (292013): Feature LogicLock is only available with a valid subscription license. You can purchase a software subscription to gain full access to this feature:* Utilizzando una licenza gratuita per studenti, non è possibile sfruttare la funzionalità LogicLock, che consente di assegnare parti specifiche del design a regioni fisiche della FPGA per controllare implementazione e timing. Il warning può essere ignorato.

8. COMMENTI E CONCLUSIONI GENERALI SUL PROGETTO

L'adozione di metodologie di progetto sincrone, basate su un unico dominio di clock e sull'uso sistematico di segnali di abilitazione, ha consentito di ot-

¹⁸ Il codice utilizzato proviene da una fonte esterna, vedi "Riferimenti bibliografici"^[2]

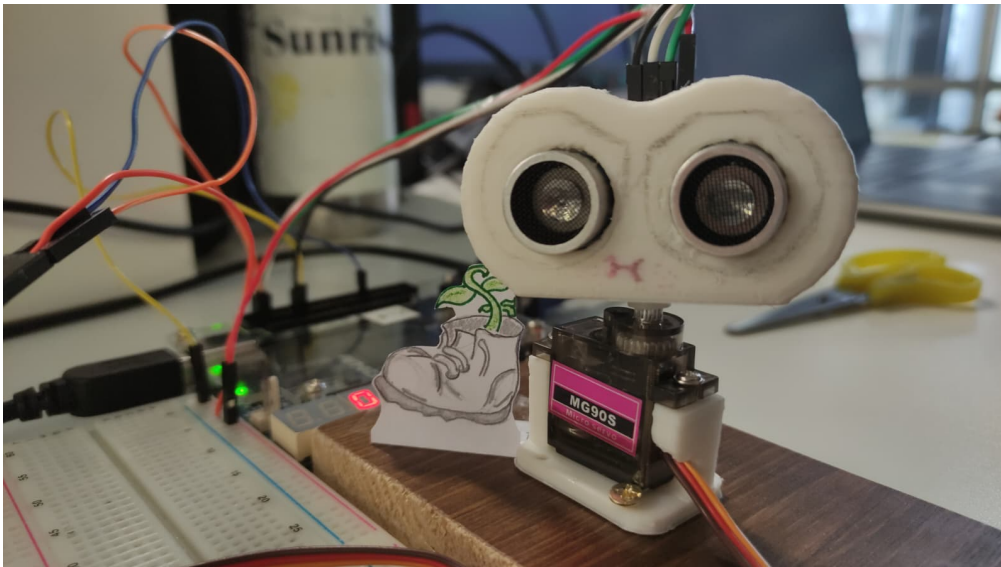


Figura 11 Prototipo.

tenere un'architettura ordinata, facilmente verificabile e robusta dal punto di vista temporale. Particolare attenzione è stata posta alla gestione dei segnali asincroni esterni, sincronizzati mediante sincronizzatori ottimi, al fine di ridurre il rischio di metastabilità e garantire un comportamento deterministico del sistema.

Le fasi di sintesi, fitting e analisi temporale hanno confermato la corretta mappatura dell'architettura sul dispositivo target, evidenziando un utilizzo contenuto delle risorse logiche e il rispetto dei vincoli di timing imposti. L'analisi dei warning generati dagli strumenti di sviluppo non ha inoltre messo in luce criticità tali da compromettere il funzionamento del sistema.

Nonostante il corretto funzionamento del sistema, sono state evidenziate possibili migliorie implementabili. Senza un feedback dalla VGA al motore si genera uno sfasamento nel tempo, poiché il motore impiega una frazione di secondo per fermarsi e invertire il verso di rotazione a 0° e 180° , mentre la VGA riparte immediatamente, causando una rapida perdita dell'allineamento angolare. Si potrebbe

inoltre ridurre la dimensione delle LUT per seno e coseno, memorizzando i valori di un solo quadrante e sfruttando le proprietà di simmetria. La gestione del segnale Trig potrebbe essere modificata¹⁹ in modo da ottenere un maggior numero di misure valide a parità di velocità di rotazione. Infine, il filtro mediano potrebbe essere ottimizzato con il fine di diminuire il tempo di computazione²⁰.

RIFERIMENTI BIBLIOGRAFICI

- [1] Altera. *Quartus® Prime Design Software*. URL: <https://www.altera.com/products/development-tools/quartus>.
- [2] AmeerAbdelhadi. *Binary-to-BCD-Converter*. URL: <https://github.com/AmeerAbdelhadi/Binary-to-BCD-Converter/blob/master/bin2bcd.v>.
- [3] Handson Technology. *HC-SR04 Ultrasonic Sensor Module User Guide*. URL: https://www.handsontec.com/dataspecs/HC-SR04-Ultrasonic.pdf?utm_source=chatgpt.com.
- [4] Terasic. *DE10-Lite Development and Education Board*. URL: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=234&No=1021#contents>.
- [5] Terasic. *DE10-Lite User Manual*, 2017.

¹⁹ Si potrebbe ridurre il tempo di attesa tra un impulso ed il successivo, nei limiti del consentito.

²⁰ Iniziando i confronti tra i dati che progressivamente riempiono il registro a scorrimento *Serial-In-Parallel-Out*